GitHub Actions

Full Assignment Submission

Part 1: True or False • Part 2: Hands-on Challenge

Student	Sylvain Kamdem
Repository	sylvain-skamdem / mygitactions
Workflow	.github/workflows/output-passing.yml
Run #	Yaml Output Passing #2 (commit: 4c0e1a0)
Date	May 2, 2026

Part 1 — True or False

Evaluate the following statements about GitHub Markdown (README.md) and GitHub Actions.

#	Statement	Answer	Explanation
1	In a README.md file, you must use HTML to create tables.	FALSE	Markdown has its own table syntax using | pipes and - dashes. HTML tables work too, but they are not required.
2	The ![alt text](image-url) syntax is the correct way to embed images in GitHub Markdown.	TRUE	This is the standard Markdown image syntax. The ! prefix distinguishes it from a regular link.
3	GitHub automatically renders README.md as the main landing page of a repository.	TRUE	GitHub detects the README.md in the root (or /docs or /.github folder) and displays it on the repo home page.
4	You can use emojis in Markdown by typing their shortcodes like :rocket:.	TRUE	GitHub Markdown supports emoji shortcodes. :rocket: renders as the rocket emoji directly in the file.
5	Badges (shields.io) in a README are only for decoration and have no functional purpose.	FALSE	Badges display live data (build status, version, license, downloads) and often link to external resources.
6	In GitHub Actions, a workflow file must be placed in the .github/workflows/ folder.	TRUE	GitHub only recognises workflow YAML files in this exact directory. Files placed elsewhere are ignored.
7	A single GitHub Actions workflow can only contain one job.	FALSE	A workflow can contain multiple jobs. They run in parallel by default, or sequentially when using needs:.
8	Jobs in a workflow always run in parallel by default.	TRUE	Without a needs: dependency, all jobs in a workflow start at the same time on separate runners.
9	You can pass data between jobs using outputs from a previous job.	TRUE	A job declares outputs: at the job level, a step saves values to $GITHUB_OUTPUT, and downstream

			jobs read them with ${{ needs.<job>.outputs.<name> }}.
10	The needs keyword is used to make one job depend on another job finishing first.	**TRUE**	needs: creates a sequential dependency. The job will not start until all listed jobs complete successfully.

Key takeaways from Part 1

- Markdown tables use | pipe and - dash syntax — HTML is optional, not required
- README.md is auto-rendered by GitHub as the repository landing page
- Badges from shields.io carry live data (build status, version, etc.) and are functional
- Workflow files only work when placed in .github/workflows/ — nowhere else
- Jobs run in parallel by default; needs: is what makes them run in sequence
- Step outputs + $GITHUB_OUTPUT + needs.<job>.outputs is the full data-passing chain

Part 2 — Hands-on Challenge

Job Dependencies, Outputs & Data Passing Between Jobs

1. Objective

The goal of this challenge was to build a GitHub Actions workflow from scratch that demonstrates three specific concepts:

- Job dependencies — making one job wait for another before it starts
- Step outputs — capturing data inside a step and saving it for later use
- Passing data between jobs — reading the outputs of one job inside a completely different job

2. How the Workflow is Triggered

The workflow runs automatically in three situations:

push to hotfix/**	Any push to a branch whose name starts with hotfix/ (e.g. hotfix/fix-login)
pull_request to main	When a Pull Request is opened or updated targeting the main branch
workflow_dispatch	A manual 'Run workflow' button appears in the GitHub Actions tab — useful for testing

```yaml
on:
  push:
    branches:
      - "hotfix/**"
  pull_request:
    branches:
      - "main"
  workflow_dispatch:
```

3. Job Structure

The workflow contains three jobs that run in strict order, one after the other:

Job 1	generate-info — Captures the current date/time and commit SHA, then exports them as outputs
Job 2	consume-info — Waits for Job 1, then reads and displays the outputs
Bonus	final-report — Waits for Job 2, then prints a one-line completion summary

Execution order: `generate-info` → `consume-info` → `final-report`

4. Job 1 — generate-info

What it does

This job runs two steps. The first step prints a start message. The second step captures the current date/time and the short commit SHA, then saves them as outputs using the modern GITHUB_OUTPUT method.

Why the id: generate label matters

The step that saves outputs has the label `id: generate`. This label is what allows other jobs to reference its outputs. Without it, the outputs exist but no other job can find them.

Full Job 1 code

```
generate-info:
  runs-on: ubuntu-latest
  outputs:
    current_date: ${{ steps.generate.outputs.current_date }}
    commit_sha: ${{ steps.generate.outputs.commit_sha }}
  steps:
    - name: Print starting message
      run: echo "Generating repository information..."
    - name: Capture date and commit SHA
      id: generate
      run: |
        CURRENT_DATE=$(date -u '+%Y-%m-%d %H:%M:%S UTC')
        COMMIT_SHA="${GITHUB_SHA::7}"
        echo "current_date=${CURRENT_DATE}" >> "$GITHUB_OUTPUT"
        echo "commit_sha=${COMMIT_SHA}" >> "$GITHUB_OUTPUT"
```

5. Job 2 — consume-info

What it does

This job reads the outputs from Job 1 and prints them in a formatted report. It also re-exports the same values so the bonus job can use them.

How needs: creates a job dependency

The `needs: generate-info` line tells GitHub Actions: do not start this job until `generate-info` has finished successfully. If Job 1 fails, Job 2 is automatically skipped.

Full Job 2 code

```
consume-info:
  runs-on: ubuntu-latest
  needs: generate-info
  outputs:
    current_date: ${{ needs.generate-info.outputs.current_date }}
    commit_sha: ${{ needs.generate-info.outputs.commit_sha }}
  steps:
    - name: Display info from job 1
      run: |
        echo "  Date & Time: ${{ needs.generate-info.outputs.current_date }}"
        echo "  Commit SHA : ${{ needs.generate-info.outputs.commit_sha }}"
        echo "  Data was successfully passed from Job 1!"
```

6. Bonus Job — final-report

```
final-report:
  runs-on: ubuntu-latest
  needs: consume-info
  steps:
    - name: Print final summary
      run: |
        echo "Workflow completed successfully on ${{ needs.consume-
info.outputs.current_date }} using commit ${{ needs.consume-
info.outputs.commit_sha }}"
```

7. Bug Found and Fixed During Testing

Bug 1 — Date & Time was blank in consume-info

The output key in Job 1 was named `current-date` (hyphen) but referenced everywhere as `current_date` (underscore). GitHub treats these as different names, so the value came back empty.

Broken	current-date: ${{ steps.generate.outputs.current_date }}
Fixed	current_date: ${{ steps.generate.outputs.current_date }}
Why	The key name on the left must exactly match what is used in needs.generate-info.outputs.KEY

Bug 2 — final-report printed on 4 separate lines

Broken	Four separate echo statements — each word on its own line
Fixed	Single echo with the full sentence in one set of quotes
Why	Each echo is its own print call. Everything must be on one echo line to print as one sentence

8. Run Results

Job	Result
`generate-info`	✓ Succeeded — date and commit SHA captured and saved as outputs
`consume-info`	✓ Succeeded — full report printed with Date & Time and Commit SHA visible
`final-report`	✓ Succeeded — single-line completion message with both values

9. Key Concepts Learned

Workflow triggers	on: push, pull_request, and workflow_dispatch control when a workflow runs
needs:	Creates a dependency — the job will not start until the listed job succeeds
id: on a step	A label that lets other jobs reference the step's outputs
outputs: on a job	Declares which step outputs the job will expose to downstream jobs
$GITHUB_OUTPUT	The modern way to save a value from a step as a reusable output
${{ needs.X.outputs.Y }}	The syntax to read another job's output inside a downstream job
GITHUB_SHA	Built-in variable — holds the full commit SHA of the current run

10. Complete Final Workflow File

```yaml
name: Yaml Output Passing


on:
  push:
    branches: ["hotfix/**"]
  pull_request:
```

```yaml
    branches: ["main"]
  workflow_dispatch:

jobs:
  generate-info:
    runs-on: ubuntu-latest
    outputs:
      current_date: ${{ steps.generate.outputs.current_date }}
      commit_sha: ${{ steps.generate.outputs.commit_sha }}
    steps:
      - name: Print starting message
        run: echo "Generating repository information..."
      - name: Capture date and commit SHA
        id: generate
        run: |
          CURRENT_DATE=$(date -u '+%Y-%m-%d %H:%M:%S UTC')
          COMMIT_SHA="${GITHUB_SHA::7}"
          echo "current_date=${CURRENT_DATE}" >> "$GITHUB_OUTPUT"
          echo "commit_sha=${COMMIT_SHA}" >> "$GITHUB_OUTPUT"

  consume-info:
    runs-on: ubuntu-latest
    needs: generate-info
    outputs:
      current_date: ${{ needs.generate-info.outputs.current_date }}
      commit_sha: ${{ needs.generate-info.outputs.commit_sha }}
    steps:
      - name: Display info from job 1
        run: |
          echo "  Date & Time: ${{ needs.generate-info.outputs.current_date
}}"
          echo "  Commit SHA : ${{ needs.generate-info.outputs.commit_sha }}"
          echo "  Data was successfully passed from Job 1!"

  final-report:
    runs-on: ubuntu-latest
    needs: consume-info
    steps:
      - name: Print final summary
        run: |
          echo "Workflow completed successfully on ${{ needs.consume-
info.outputs.current_date }} using commit ${{ needs.consume-
info.outputs.commit_sha }}"
```